

Trabalho prático em grupo – Módulo 1

Cinthia Mie Ungefehr¹[PG58422], Fabiano de Carvalho e Sousa¹[PG55213], João Pedro da Silva Barbosa¹[PG58984], and Lóren Narrane Leandro Oliveira da Silva¹[PG61985]

¹ Universidade do Minho, Largo do Paço, 4704-553, Braga

Abstract. O presente trabalho tem como objetivo a classificação de textos quanto sua origem, sendo ela humana ou de um modelo generativo. No caso de um modelo generativo, também identificar qual é a empresa responsável pelo modelo que gerou o texto em questão.

Keywords: IA vs Humano · Texto Generativo · Aprendizagem Profunda

1 Introdução

Nos últimos anos, os modelos de linguagem baseados em *Deep Learning* têm evoluído significativamente, permitindo a geração de textos coerentes e quase indistinguíveis daqueles de origem humana. Esta evolução veio com desafios, sendo um deles a identificação da origem do texto.

Este relatório foi desenvolvido no âmbito do trabalho prático do Módulo 1 da unidade curricular de Aprendizagem Profunda. O principal objetivo deste trabalho consiste no desenvolvimento de modelos de *Machine / Deep Learning* capazes de distinguir a origem de textos, nomeadamente entre textos gerados por diferentes modelos de Inteligência Artificial e textos produzidos por seres humanos.

Inicialmente, foi construído um *dataset* para ser utilizado no treino dos modelos. Para tal, foram recolhidos dados em língua inglesa a partir de diversos repositórios públicos. Após a criação do *dataset*, procedeu-se ao desenvolvimento dos modelos, cuja submissão foi dividida em três fases, cada uma contendo dois modelos.

Na primeira submissão, o foco incidiu na construção de modelos com implementação própria e em PyTorch, sem recurso a arquiteturas do tipo *transformer*. Nas segunda e terceira submissões, não existiram restrições quanto aos modelos utilizados, permitindo a experimentação de diferentes abordagens, bem como a otimização dos modelos previamente desenvolvidos.

Concluído o desenvolvimento dos modelos e obtidos os resultados das diferentes submissões, fornecidos pelo docente, procedeu-se à análise comparativa do desempenho dos mesmos. Por fim, apresentam-se as conclusões do trabalho realizado.

2 Dataset

A criação do *dataset* para treino dos modelos iniciou-se com a busca de bases com textos humanos e textos gerados por modelos de inteligência artificial de alguma das 4 empresas indicadas no enunciado do trabalho: Anthropic, Google, OpenAI e Meta. Os textos escolhidos foram restritos àqueles com formato de resumo científico ou artigo em busca de selecionar textos mais semelhantes aos exemplos disponibilizados pelo professor.

Após uma pequena avaliação dos textos nos *datasets* encontrados, quatro foram selecionados para compor a base de dados a ser utilizada neste trabalho: human-AI-generatedTextCorpus [1], GPT vs. Human: A Corpus of Research Abstracts [2], AI vs Human Scientific Text Dataset (AIGTxt) [3] e Human vs. LLM Text Corpus [4].

Os dados recolhidos passaram por um breve tratamento:

- Criação da coluna “Label”;
- Renomeação das colunas para se adequarem ao formato definido pelo professor;
- Remoção de caracteres indesejados e que não eram esperados aparecer nos textos das submissões (ex. “n”);
- Remoção de textos duplicados; e
- Criação de coluna com contagem de palavras no texto (a partir da submissão 2).

Por fim, as quatro bases foram unidas numa única tabela e exportadas de três maneiras:

- custom_dataset_v1.csv: tabela com 668.096 registros (2GB).
 - Human: 353.470 textos;
 - OpenAI: 125.852 textos;
 - Meta: 122.773 textos;
 - Google: 55.696 textos; e
 - Anthropic: 10.305 textos.

Dataset bastante desbalanceado. O tamanho elevado causou muita lentidão para leitura dos dados e treino dos modelos;

- custom_dataset_v1_mini.csv: tabela com 210.305 registros (474MB).
 - Human: 50.000 textos;
 - OpenAI: 50.000 textos;
 - Meta: 50.000 textos;
 - Google: 50.000 textos; e
 - Anthropic: 10.305 textos.

Dataset menos desbalanceado que a versão original. O tamanho menor permitiu o treino de modelos em tempo hábil;

- custom_dataset_v2_mini.csv: tabela com 170.915 registros (93MB).
 - Human: 70.610 textos;
 - OpenAI: 30.000 textos;
 - Meta: 30.000 textos;
 - Google: 30.000 textos; e
 - Anthropic: 10.305 textos.

Dataset com restrição para textos de até 120 palavras e com balanceamento de classes mais próximo aos *datasets* de exemplo e das submissões.

3 Modelos

3.1 Modelos submetidos

Modelo submissão 1 A Este modelo focou-se na implementação manual de uma rede neural utilizando exclusivamente as bibliotecas NumPy e Pandas, tendo por base o código fornecido nas aulas 2 e 3. Foram realizadas as devidas adaptações de forma a adequar a implementação aos requisitos específicos do problema.

No pré-processamento dos dados, foi efetuada uma limpeza textual que incluiu a conversão de todos os caracteres para minúsculas e a remoção de pontuação. Adicionalmente, o conjunto de dados foi filtrado de forma a incluir apenas os textos com comprimento entre 80 e 120 palavras, garantindo maior uniformidade entre as amostras.

A representação dos textos foi efetuada através da técnica *Bag-of-Words* (BoW), com um vocabulário limitado às 10.000 palavras mais frequentes.

A arquitetura da rede neuronal foi definida de forma sequencial, sendo composta pelas seguintes camadas:

- Camada de entrada: recebe o vetor BoW resultante do pré-processamento
- Camada densa 1: 128 neurónios, função de ativação ReLU e regularização L2
- Camada densa 2: 64 neurónios, função de ativação ReLU
- Camada de saída: 5 neurónios com função de ativação Softmax, correspondentes às diferentes classes do problema

Como forma de avaliar a eficácia da arquitetura proposta, foi implementado um modelo baseline baseado em Regressão Logística. Este modelo consiste numa arquitetura de camada única (*Single-layer Perceptron*), que liga diretamente a entrada aos 5 neurónios de saída com função de ativação Softmax, sendo matematicamente equivalente a uma Regressão Logística Multinomial.

Relativamente à configuração do treino, foram utilizadas as seguintes definições:

- **Metric:** Accuracy
- **Otimização:** Gradient descent com momentum (0.8)
- **Learning rate:**
 - DNN: 0.005
 - Baseline: 0.01
- **Batch size:** 32
- **Nº de epochs:**
 - DNN: 1000
 - Baseline: 100
- **Early stopping:** 10 epochs

Quanto aos resultados, o modelo apresentou as seguintes precisões:

- **validação (DNN):** 65.32%
- **validação (baseline):** 61.91%
- **dataset_exemplos.csv (DNN):** 37,60%
- **dataset_exemplos.csv (baseline):** 32,80%
- **submissão 1 (DNN):** 34,67%

Verifica-se que o modelo DNN apresenta um desempenho superior ao modelo baseline em todos os cenários considerados, evidenciando a vantagem de arquiteturas mais profundas na captura de padrões mais complexos nos dados.

No entanto, observa-se uma diminuição significativa da performance ao passar do conjunto de validação para os datasets externos e para a submissão final. Este comportamento poderá indicar algum grau de overfitting ao dataset de treino, bem como possíveis diferenças na distribuição dos dados utilizados na validação externa.

Modelo submissão 1 B Na primeira submissão, o modelo desenvolvido foi uma rede neuronal do tipo LSTM (*Long Short-Term Memory*), bidirecional. A sua arquitetura contém uma camada de *embedding*, seguida pela LSTM bidirecional, e um bloco de classificação que inclui camadas totalmente conectadas. A sequência é: Linear, função de ativação ReLU (*Rectified Linear Unit*), uma camada de *dropout* para a regularização e, por último, uma camada Linear.

O processo de otimização foi realizado com o algoritmo Adam, enquanto a função de perda foi a CrossEntropyLoss, apropriada para problemas de classificação multiclasse. Durante a construção do *dataset*, houve um desbalanceamento entre a classe Anthropic e as restantes. Para mitigar este efeito, foram introduzidos pesos na função de perda, dando mais importância à classe minoritária durante o treino. Além disso, foi implementado um mecanismo de *early stopping* com o parâmetro de *patience* igual a 2, com o objetivo de reduzir o *overfitting* e interromper o treino quando não houver melhorias no desempenho do modelo no conjunto de validação.

Os resultados mostraram a seguinte precisão:

- **validação:** 89,30%
- **dataset_exemplos.csv:** 40,00%
- **submissão 1:** 39,33%

Estes valores indicam que o modelo tem um desempenho moderado mas que algumas melhorias são necessárias, principalmente na arquitetura e no pré-processamento dos dados.

Modelo submissão 2 A Nesta etapa do projeto foi permitido usar modelos do tipo *Transformer*, o que nos levou a decisão de fazer dois modelos em que o primeiro serviria de base de comparação para o segundo. Deste modo, o modelo 2 A é um *Transformer* desenvolvido em *PyTorch* a partir do código apresentado na aula 5 do curso, sem uso de *transfer learning*, enquanto o modelo 2 B faz uso.

Como o problema de classificação proposto neste projeto é mais complexo do que o que o visto em aula, alguns ajustes foram aplicados no tamanho do modelo e em seus hiperparâmetros. Quanto a estes, neste momento ainda não foi introduzida uma metodologia de busca automática na otimização, sendo seus valores obtidos manualmente em tentativas consecutivas para melhora da acurácia de validação.

Em comparação com o código de aula, o modelo 2 A ao fim apresenta as seguintes alterações:

- **embed_dim**: 512
 - **num_heads**: 8
 - **ff_dim**: 2048
 - **dropout_rate**: 30%
 - **learning rate**: 0.00025
 - **otimizações**: *early stopping* (parada na *epoch 8*)
- Os resultados de referência deste modelo foram as seguintes precisões:
- **validação**: 85.06%
 - **dataset_exemplos.csv + submissão 1 (revelado)**: 46,22%
 - **submissão 2**: 34,00%

Modelo submissão 2 B Na etapa intermediária do desenvolvimento, foi decidido explorar uma abordagem baseada em *Transformers*, utilizando o modelo pré-treinado DistilBERT. Esta abordagem introduz o conceito de *transfer learning* e representações contextuais mais avançadas.

Este modelo utiliza um *tokenizer* pré-treinado associado ao modelo DistilBERT. Este *tokenizer* é capaz de segmentar automaticamente o texto em *tokens* e, além disso, são geradas *attention masks*, que permitem ao modelo distinguir entre a informação relevante e os elementos de preenchimento. O modelo DistilBERT é carregado com pesos previamente treinados e, posteriormente, é ajustado para a tarefa específica de classificação.

O processo de otimização foi similar ao da submissão 1, no entanto, foi implementado também um *learning rate scheduler* com uma estratégia de *warmup* linear, que aumenta gradualmente a taxa de aprendizagem numa fase inicial e decresce posteriormente, resultando numa maior estabilidade e convergência.

Quanto aos resultados, o modelo apresentou as seguintes precisões:

- **validação**: 85.64%
- **dataset_exemplos.csv**: 51,20%
- **submissão 1 (revelado)**: 49,00%
- **submissão 2**: 38,00%

Estes resultados demonstram uma melhoria significativa relativamente à abordagem anterior.

Modelo submissão 3 A A grande diferença na implementação desta terceira submissão foi a utilização de *embeddings* pré-treinados GloVe (*Global Vectors*

for Word Representation). Estes dão-nos representações vetoriais densas das palavras, capturando relações semânticas entre elas. No início do modelo, cada palavra do vocabulário é associada ao vetor GloVe, se disponível. Os restantes são inicializados aleatoriamente. Ao contrário de outras abordagens, estes *embeddings* são ajustados durante o treino, o que ajuda o modelo a adaptar-se ao problema específico.

O modelo é baseado em GRU (*Gated Recurrent Unit*) bidirecional com várias camadas. Isto ajuda a capturar sequências no texto de forma eficiente. Para melhorar a representação das sequências, combinamos três estratégias: *average pooling*, *max pooling* e, um mecanismo de atenção. O mecanismo de atenção aprende a dar pesos diferentes às palavras, destacando as mais importantes. As três representações são depois concatenadas e passam por uma camada de *dropout* para regularização e, finalmente, por uma camada Linear para a classificação final.

Quanto à otimização, foram utilizados os mesmos métodos anteriores, função de perda CrossEntropyLoss e algoritmo de otimização AdamW, com regularização adicional. No entanto foi aplicado também *gradient clipping* juntamente do previamente utilizado *Learning Rate Scheduler*, para melhorar a estabilidade e desempenho.

Nesta última submissão os resultados obtidos foram os seguintes:

- **validação:** 82,00%
- **dataset_exemplos.csv:** 48,00%
- **submissão 2 (revelado):** 39,00%
- **submissão 3:** 32,67%

que demonstra uma pequena subida de precisão de 1% na submissão 2. No entanto, é visível também uma pequena descida no dataset_exemplos.

Modelo submissão 3 B Durante a apresentação do trabalho, foi colocado como passo futuro a tentativa de criação de uma pipeline que consistisse de um modelo para classificar se um texto foi gerado por uma IA ou não e, caso tenha sido, a qual empresa pertence o modelo generativo em questão.

Como primeiro teste foram criados dois modelos de Regressão Logística para definir uma *baseline*.

O primeiro modelo obteve acurácia de 0.75 no teste com o dataset_v2_mini. Já o segundo modelo obteve 0.73 de acurácia no teste. Contudo, a pipeline obteve apenas 0.34 de acurácia na validação com os *datasets* das submissões + o *dataset* de exemplos disponibilizados com forte propensão a classificar os textos como “Human” (label 2), vide Fig. 1.

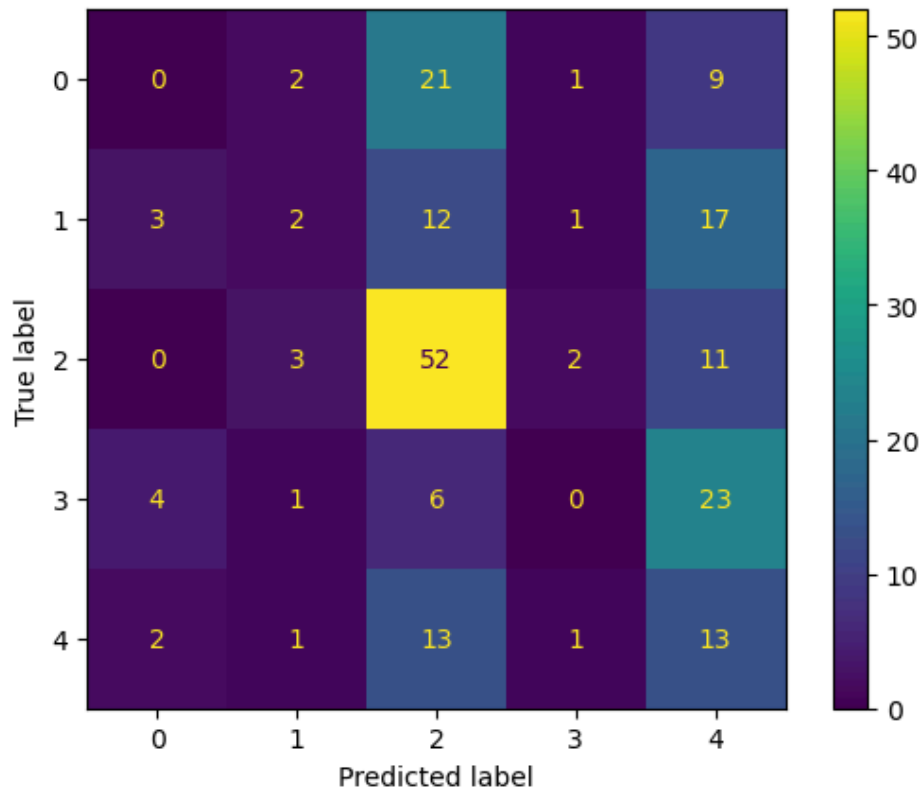


Fig. 1. Matriz de Confusão de Validação

Dados os maus resultados, mas agora com uma *baseline* de referência iniciou-se o processo para criação de um modelo que fizesse uso de técnicas mais robustas, como modelos pré-treinados. Infelizmente, não foi possível concluir tais modelos, sendo a versão com as Regressões Logísticas a enviada na submissão.

3.2 Modelos testados

Modelo RNN vs LSTM vs GRU Com o objetivo de replicar o comparativo de desempenho dos 3 tipos de modelos vistos na 4ª aula do curso, foram aplicados os 3 modelos RNN, LSTM e GRU sobre o *dataset* custom_dataset_v1_mini.csv .

Por questões de performance, o exercício foi simplificado para as seguintes condições:

- o tamanho do vocabulário foi limitado a apenas 20 mil palavras;
- o *dataset* para treino e validação foi reduzido para 10.305 sentenças para cada uma das 5 classes.

Além disso, a fim de uniformizar a comparação entre modelos, foram adotados exatamente os mesmos hiperparâmetros:

- divisão balanceada do *dataset* reduzido em 80% para treino e 20% para validação;
- a quantidade de treino fixa em 10;
- tamanho do *embedding* em 100;
- 2 camadas recorrentes empilhadas;
- cada camada com 100 estados ocultos;
- learning rate fixo de 0.001, sem ajustes dinâmicos.

O código dos modelos pode ser visto no github do grupo, em MMC_AP_Trabalho/Dev/PyTorch_Alt/GRU_pytorch.ipynb .

Os resultados levantados tiveram dois propósitos:

1. ajudar a decidir por qual modelo aplicar na primeira submissão; e
2. testar os efeitos no tratamento inicial sobre o *dataset* `custom_dataset_v1_mini.csv`, ao se selecionar preferencialmente sentenças cujo vocabulário estivessem mais correlacionadas ao *dataset-exemplos.csv* .

As classificações obtidas por estes modelos acabaram por não serem submetidos para avaliação um vez que o treino foi sobre o *dataset* reduzido e não se aplicou otimização nos hiperparâmetros. Seus resultados de acurácia e de média de função de perda podem ser visto na figura Fig. 2.

Qualitativamente pode-se observar o mesmo comportamento visto no exemplo de aula, com o *dataset* “ag_news”, no qual o modelo RNN apresentou resultado de acurácia na base de validação significativamente inferior aos modelos LSTM e GRU. Estes apresentaram desempenho de validação próximos entre si, em torno de 81% de acurácia.

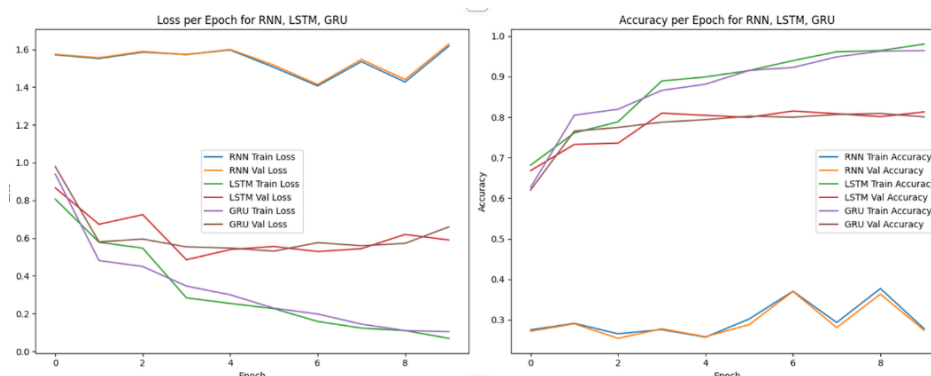


Fig. 2. Comparativo de performance dos modelos RNN vs LSTM vs GRU

Por fim, para fins de comparação com o modelo utilizado para a submissão 1B, foi treinado mais uma vez apenas o modelo GRU deste exercício, utilizando um vocabulário de 50 mil palavras e um balanceamento de classes para seleção das amostras de *custom_dataset_v1_mini.csv* com 3 vezes mais amostras da classe Human, a fim de aproximar a distribuição da utilizada pelo professor em *dataset-exemplos.csv*. A melhora de resultado de acurácia obtida foi expressiva, atingindo 84,2% de acurácia na base de validação já na terceira *epoch*.

4 Resultados obtidos

Os resultados obtidos pelos modelos enviados para cada submissão são descritos na tabela a seguir, dos quais os *datasets* de validação são diferentes entre os modelos, pois além do *dataset-exemplos.csv*, para cada nova submissão, também se incluiu as *labels* disponibilizadas da submissão anterior.

Table 1. Tabela de Resultados

Modelo	Treino	Teste	Validação	Ranking
Baseline de Regressão Logística	68.74%	61.91%	32.80%	N/A
DNN (Subm 1A)	87.88%	65.32%	34.67%	13
LSTM (Subm 1B)	89.30%	78.98%	39.33%	10
DistilBert (Subm 2A)	87.90%	86.31%	38.00%	17
Transformer (Subm 2B)	86.13%	85.06%	34.00%	16
GloVe (Subm 3A)	99.36%	82.00%	39.89%	22
Regressão Logística duplo (Subm 3B)	N/A	73.00%	34.00%	21

É possível observar que os modelos pré-treinados e de aprendizagem profunda apresentaram, de forma geral, melhores resultados. Apesar da maior complexidade associada, a diferença desses modelos face aos modelos de Regressão Logística revela-se relativamente reduzida, indicando que os ganhos de desempenho não foram tão significativos quanto inicialmente esperado.

De forma geral, os modelos baseados em transformers e embeddings pré-treinados apresentaram melhor capacidade de generalização, especialmente em datasets externos. No entanto, verificou-se que o aumento da complexidade nem sempre resultou em ganhos significativos de desempenho, sugerindo que a qualidade e distribuição dos dados têm um papel determinante na performance final dos modelos.

Estes resultados reforçam a importância da qualidade e representatividade dos dados, sugerindo que melhorias no dataset poderão ter um impacto mais significativo do que o aumento da complexidade dos modelos.

5 Conclusão

Este trabalho permitiu compreender a importância do pré-processamento e da construção do dataset, que se revelou um dos fatores mais determinantes no desempenho dos modelos. Adicionalmente, verificou-se que modelos mais complexos, como transformers, apresentam maior capacidade de generalização, embora dependam fortemente da qualidade dos dados de treino.

As métricas finais obtidas também não podem ser consideradas aceitáveis, estando muito abaixo do que seria ideal, já que numa situação real de uso, seria impossível confiar em qualquer um dos modelos.

Teoriza-se que os resultados não são uma conclusão necessariamente da arquitetura criada para os modelos, mas sim uma consequência dos dados escolhidos, pois é bastante possível que os textos não reflitam um mesmo formato ou teor que os textos disponibilizados nas submissões.

Assim, algum trabalho futuro faz-se necessário para criação de um *dataset* de treino mais fiel à realidade de teste e um novo treino dos modelos já criados ou até mesmo a criação de novos modelos.

References

1. LorenzM97: human-AI-generatedTextCorpus, <https://github.com/LorenzM97/human-AI-generatedTextCorpus/tree/main>, last accessed 2026/04/06.
2. Eriksen, H.: GPT vs. Human: A Corpus of Research Abstracts, <https://www.kaggle.com/datasets/heleneeriksen/gpt-vs-human-a-corpus-of-research-abstracts?resource=download>, last accessed 2026/04/06.
3. Arbab, B.A.: AI vs Human Scientific Text Dataset (AIGTxt), <https://www.kaggle.com/datasets/barkataliarbab/ai-vs-human-scientific-text-dataset>, last accessed 2026/04/06.
4. Grinberg, Z.: Human vs. LLM Text Corpus, <https://www.kaggle.com/datasets/starblasters8/human-vs-llm-text-corpus?select=data.csv>, last accessed 2026/04/06.